IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.0429000

An Empirical Evaluation of NVIDIA Kumo Tabular Against Traditional Machine Learning Models on Small Tabular Datasets

MD. NOYON AHMED JOY
Independent Researcher, Bangladesh (ORCID iD: 0009-0004-9225-4610, https://orcid.org/0009-0004-9225-4610)
Corresponding author: MD. Noyon Ahmed Joy (e-mail: noyonahmedjoyml@gmail.com).

This work did not receive any external funding.

ABSTRACT Tabular foundation models predict the labels of new rows from labelled rows in a single forward pass, without training on the target dataset. This paper reports a controlled, reproducible comparison of NVIDIA Kumo Tabular, a recently released pretrained tabular transformer, with logistic or linear regression, random forests and histogram-based gradient boosting on two small public datasets: breast cancer diagnosis (569 rows, classification) and diabetes progression (442 rows, regression). All models share five random train/test splits and default settings, and are evaluated with standard metrics, runtime measurements, a paired statistical analysis that accounts for overlapping training sets, and a training-size study from 25 rows upwards. The smallest Kumo Tabular checkpoint obtained the best mean on every metric: accuracy 0.989 versus 0.977 for the strongest baseline, malignant-class recall 0.976 versus 0.948, and coefficient of determination 0.503 versus 0.496. In classification it matched or exceeded logistic regression on all five splits for every metric, but after correction for split overlap and multiple comparisons no difference is statistically significant at the 5% level, and in regression the gap is within split-to-split variation. Kumo Tabular was markedly more robust than linear regression with 25 to 100 training rows, yet offered no advantage over logistic regression in classification at those sizes. Its prediction time per split was about 6 to 220 times that of the linear models across the hardware and runs measured. Kumo Tabular is a strong default for small tables, but on these benchmarks its advantage over well-chosen classical models is modest.

INDEX TERMS Benchmarking, foundation models, gradient boosting, in-context learning, Kumo Tabular, small data, TabPFN, tabular data

I. INTRODUCTION

MOST machine learning in industry and science is applied to tabular data, and for this setting tree ensembles and regularised linear models have remained the practical default for many years. Extensive comparisons found that gradient-boosted trees match or outperform deep neural networks on typical medium-sized tables [1]–[3], although the gap depends strongly on the dataset [4] and on tuning [5].

Tabular foundation models change how a model is obtained. Instead of fitting parameters to the training split of each new dataset, a transformer is pretrained once on a large collection of tables and then conditions on the labelled rows of a new table at prediction time. TabPFN [6], [7] showed that a network pretrained on synthetic data can approximate Bayesian prediction for small tables, and later models extended the idea to larger inputs [8]. NVIDIA Kumo Tabular [9], [10] is a recent, openly released model of this type. It is distributed through the `structured-data-models` library [11] in three sizes and, according to its developers, was pretrained only on synthetic tables.

Published claims about such models usually come from large benchmark suites evaluated by the model developers. Practitioners who consider adopting a pretrained tabular model for a small dataset face narrower questions: does it outperform the simple models they would otherwise use, is the difference larger than the noise of a small test set, and what does it cost? This paper addresses these questions for Kumo Tabular on two well-known datasets. It makes the following contributions:

- A reproducible comparison of Kumo Tabular with three classical baselines on identical splits, including classification, regression, a training-size study and runtime

measurements. Code, results and notebooks are public [12], [13].
- A paired statistical analysis that accounts for the dependence between resampled splits [14] and for multiple comparisons [15], together with an explicit statement of what five splits can and cannot establish.
- An analysis of where the pretrained model helps (classification recall, very small regression training sets, built-in predictive intervals) and where it does not (small classification training sets, the full regression task, prediction cost).

The study is deliberately small in scope. It does not attempt a general ranking of tabular methods, and the medical datasets are used as standard benchmarks only; nothing in this paper should be read as evidence of clinical reliability.

II. RELATED WORK

A. CLASSICAL MODELS FOR TABULAR DATA

Logistic regression [16] and ordinary least squares remain strong baselines for small, low-dimensional problems. Random forests [17] average decorrelated decision trees, and gradient boosting [18] fits trees sequentially to the residuals of the current ensemble; efficient implementations such as XGBoost [19], LightGBM [20] and CatBoost [21] made boosting the default for many tabular tasks. Scikit-learn [22] provides the implementations used in this study, including a histogram-based boosting estimator in the style of LightGBM.

B. DEEP LEARNING ON TABLES

Neural architectures designed for tables, such as TabNet [23] and transformer variants [24], have been compared extensively with tree ensembles. Surveys and benchmarks report that trees remain competitive or superior on many medium-sized datasets [1]–[3], that well-regularised multilayer perceptrons can close the gap [5], and that the best family varies across datasets, with dataset properties mattering more than the choice between neural networks and boosted trees [4].

C. IN-CONTEXT LEARNING

In-context learning describes a pretrained model solving a new task from examples supplied in its input, without updating its weights. It was popularised by large language models [25], which build on the transformer architecture [26], and has been shown to emerge for simple function classes when transformers are trained on suitable task distributions [27]. Large language models have also been applied directly to serialised tables [28].

D. TABULAR FOUNDATION MODELS

Prior-data fitted networks (PFNs) [29] are transformers trained on datasets sampled from a prior so that a single forward pass approximates the posterior predictive distribution. TabPFN [6] applied this to small classification tables using priors based on structural causal models, and its successor [7] added regression, larger inputs and a two-way attention over rows and columns. TabICL [8] compresses each row into a fixed-size embedding with column- and row-wise attention before in-context learning, which allows much larger tables. Set-based attention with inducing points [30] is a common building block for processing columns efficiently.

E. NVIDIA KUMO TABULAR

Kumo Tabular [9], [10] follows this line of work. Its developers describe it as a transformer with column, row and in-context attention "as introduced in TabICL and TabPFN", pretrained entirely on synthetic tables sampled from structural causal models, released in three sizes with 28M to 215M parameters under the OpenMDW 1.1 licence. They report first places on several public benchmark suites [10]; those claims are the developers' own and are not evaluated here. To the best of the author's knowledge, no independent evaluation of Kumo Tabular on these two datasets had been published at the time of writing.

III. BACKGROUND: TRAINING VERSUS IN-CONTEXT LEARNING

Let $\mathcal{D}_{\mathrm{tr}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ be a labelled training set and $\mathbf{x}_q$ a query row.

A. CONVENTIONAL SUPERVISED LEARNING

A classical model chooses parameters by (regularised) empirical risk minimisation on the dataset at hand,

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i\big) + \lambda\, \Omega(\boldsymbol{\theta}), \tag{1}$$

and predicts $\hat{y}_q = f_{\hat{\boldsymbol{\theta}}}(\mathbf{x}_q)$. The cost of (1) is paid once per dataset; afterwards predictions are cheap and independent of n.

B. PRETRAINED IN-CONTEXT LEARNING

A tabular foundation model has parameters $\boldsymbol{\phi}$ that were fitted once, before the target dataset was seen, by minimising the expected predictive loss over many datasets $\mathcal{D}$ drawn from a generator $p(\mathcal{D})$:

$$\boldsymbol{\phi}^{\star} = \arg\min_{\boldsymbol{\phi}} \mathbb{E}_{\mathcal{D}\sim p(\mathcal{D})}\Big[-\log q_{\boldsymbol{\phi}}\big(y_q \mid \mathbf{x}_q, \mathcal{D}_{\mathrm{ctx}}\big)\Big], \tag{2}$$

where each sampled dataset is split into context rows $\mathcal{D}_{\mathrm{ctx}}$ and held-out query rows. For a sufficiently expressive model, $q_{\boldsymbol{\phi}}$ approximates the posterior predictive distribution $p(y_q \mid \mathbf{x}_q, \mathcal{D}_{\mathrm{ctx}}) = \int p(y_q \mid \mathbf{x}_q, \psi)\, p(\psi \mid \mathcal{D}_{\mathrm{ctx}})\, d\psi$ implied by the generator [29]. On a new dataset, $\boldsymbol{\phi}^{\star}$ stays fixed and the training split is supplied as context:

$$\hat{p}(y_q \mid \mathbf{x}_q) = q_{\boldsymbol{\phi}^{\star}}\big(y_q \mid \mathbf{x}_q, \mathcal{D}_{\mathrm{tr}}\big). \tag{3}$$

No gradient step is taken on $\mathcal{D}_{\mathrm{tr}}$. Everything the model "learns" about the dataset happens inside the forward pass, through attention from query rows to context rows. Fig. 1 contrasts the two procedures. Two practical consequences follow. First, there is no separate training stage to tune or validate. Second, every prediction call must process the context again;

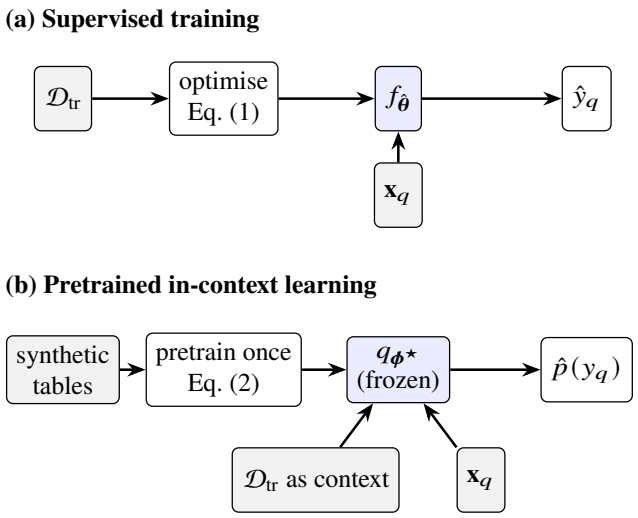


FIGURE 1. Conventional supervised learning fits parameters to each dataset (a). A tabular foundation model is pretrained once on synthetic datasets and conditions on the labelled rows of a new dataset at prediction time, without updating its weights (b).

there is no compact fitted model that can be applied later at negligible cost, unless context representations are cached.

C. KUMO TABULAR AS USED IN THIS STUDY

The following description is taken from the developers' documentation [10] and the source code of `structured-data-models` 0.2.1 [11]. Cell values are embedded with learned Fourier features and grouped into tokens. A row encoder alternates column-wise induced self-attention, in which every row attends only to context rows, with row-wise attention across the tokens of one row; four readout tokens summarise each row into a fixed-size embedding. A final transformer performs in-context learning over row embeddings, with context rows attending to each other and query rows attending to context rows only. The classification head outputs class scores for up to ten classes; the library wraps it in an error-correcting output code scheme [31]. The regression head outputs 999 quantiles q_τ, $\tau \in \{0.001, \ldots, 0.999\}$ [32].

A built-in preprocessing recipe creates an ensemble of E views of the table. Each view standardises the features and applies one of four transformations (identity, power transform, robust scaling with soft clipping, or rank-Gaussian), shuffles the column order, and, for classification, cyclically shifts the class labels. For classification, the estimator outputs $\mathbf{z}_e$ are averaged and normalised,

$$\hat{\mathbf{p}} = \operatorname{softmax}\Big(\frac{1}{E}\sum_{e=1}^{E}\mathbf{z}_e\Big), \qquad \hat{y} = \arg\max_k \hat{p}_k; \tag{4}$$

for regression, each estimator's quantiles are sorted and combined with a 20% trimmed mean. This study uses the `small` checkpoints (27.5M parameters for classification and 28.5M for regression, counted from the loaded models) with $E = 8$.

TABLE 1. Datasets used in the study.

	Breast cancer [33]	Diabetes [34]
Task	binary classification	regression
Rows	569	442
Features	30 numerical	10 numerical
Target	212 malignant, 357 benign	progression, 25 to 346
Train / test rows	455 / 114	353 / 89
Split	stratified 80/20	random 80/20

IV. METHODOLOGY

A. DATASETS

Both datasets ship with scikit-learn and are summarised in Table 1. The Breast Cancer Wisconsin (Diagnostic) dataset [33] contains 30 numerical features computed from digitised images of fine-needle aspirates. The label was recoded so that malignant tumours form the positive class, because missed malignant cases are the costly error. The diabetes dataset [34] contains ten baseline variables and a quantitative measure of disease progression one year later; features were used in their original units rather than the default standardised version.

B. EXPERIMENTAL PROTOCOL

Each experiment is repeated over five train/test partitions generated with seeds 42 to 46 (stratified for classification). Within a seed, every model receives the identical partition. For Kumo Tabular, the training rows with their targets form the context, and the test rows without the target column form the query. The implementation rejects query tables that contain the target, checks that training and test rows are disjoint, and validates the length and finiteness of all predictions before metrics are computed. If Kumo Tabular fails to load or run, its results are recorded as incomplete rather than replaced; this did not occur in the reported runs.

C. MODELS AND CONFIGURATION

Table 2 lists the models. All baselines use scikit-learn defaults with a fixed random state; logistic regression receives standardised inputs, and random forests use 500 trees. No model is tuned, so the comparison is between out-of-the-box methods. Kumo Tabular uses the `small` checkpoint, eight ensemble members, an estimator batch size of one and a seeded random number generator, which makes its predictions deterministic for a given split. For regression, the median $q_{0.5}$ is the point prediction and $[q_{0.05}, q_{0.95}]$ a nominal 90% predictive interval.

D. EVALUATION METRICS

For classification with true positives TP, false positives FP, true negatives TN and false negatives FN, the study reports

$$\text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}, \tag{5}$$

$$F_1 = \frac{2\,\text{Precision}\cdot\text{Recall}}{\text{Precision}+\text{Recall}}, \tag{6}$$

accuracy $(TP + TN)/m$ on m test rows, the area under the ROC curve [35] and the confusion matrix. Class predictions

TABLE 2. Models and settings. All models use the same splits; none is tuned.

Model	Configuration
Kumo Tabular	`small` checkpoint, $E = 8$ estimators, seeded; median quantile for regression
Logistic Reg.	standardisation, L2 penalty $C = 1$, `max_iter`=5000
Linear Reg.	ordinary least squares
Random Forest	500 trees, default depth and features
HistGB	histogram gradient boosting, defaults (100 iterations, min. 20 samples per leaf)

TABLE 3. Computational environment.

GPU	NVIDIA GeForce GTX 980M, 8 GB (compute capability 5.2)
CPU	Intel Core i7-6700HQ, 4 cores, 8 threads
OS	Windows 10
Software	Python 3.12.16, PyTorch 2.14.1 (CUDA 12.6) [36], structured-data-models 0.2.1, scikit-learn 1.9.1
Precision	float32; float16 autocast disabled (GPU older than 7.0)

use the arg-max rule (4), equivalent to a 0.5 threshold. For regression with predictions $\hat{y}_j$,

$$\mathrm{MAE} = \tfrac{1}{m}\textstyle\sum_j |y_j - \hat{y}_j|, \quad \mathrm{RMSE} = \sqrt{\tfrac{1}{m}\textstyle\sum_j (y_j - \hat{y}_j)^2}, \tag{7}$$

$$R^2 = 1 - \frac{\sum_j (y_j - \hat{y}_j)^2}{\sum_j (y_j - \bar{y})^2}. \tag{8}$$

Interval quality is measured by the mean interval width and the empirical coverage

$$\mathrm{Coverage} = \frac{1}{m}\sum_{j=1}^{m} \mathbf{1}\left[q_{0.05,j} \le y_j \le q_{0.95,j}\right], \tag{9}$$

where $\mathbf{1}[\cdot]$ is the indicator function.

E. RUNTIME MEASUREMENT

Runtime is the wall-clock fit time plus prediction time per split. Kumo Tabular has no fitting stage, so its full cost is counted as prediction time. One warm-up call on training rows precedes timing to exclude one-off CUDA initialisation. Kumo Tabular ran on the GPU and the baselines on the CPU (Table 3); runtimes therefore compare complete pipelines on the available hardware, not algorithms on equal hardware.

F. STATISTICAL ANALYSIS

For each metric, the paired advantage of Kumo Tabular over the strongest baseline on split s is d_s, signed so that $d_s > 0$ favours Kumo for every metric. With $k = 5$ splits, mean $\bar{d}$ and standard deviation s_d, the usual paired t-test treats the d_s as independent. They are not: the training sets of different splits overlap heavily, which makes the naive test anti-conservative [37]. The study therefore also reports the corrected resampled t-statistic of Nadeau and Bengio [14],

$$t_{\mathrm{corr}} = \frac{\bar{d}}{\sqrt{\left(\frac{1}{k} + \frac{n_{\mathrm{te}}}{n_{\mathrm{tr}}}\right) s_d^2}}, \tag{10}$$

with $k - 1$ degrees of freedom and $n_{\mathrm{te}}/n_{\mathrm{tr}} \approx 0.25$, the corresponding 95% confidence interval, and Holm's step-down adjustment [15] across the metrics of each task. As a distribution-free check, the exact Wilcoxon signed-rank test [38] is given; with five non-zero differences its smallest attainable two-sided p-value is $2/2^5 = 0.0625$, so it cannot reach the 5% level in this design. Counts of wins, ties and losses are reported descriptively [39].

G. TRAINING-SIZE STUDY

To examine the small-data regime, the training split of seeds 42 to 44 is subsampled (stratified for classification) to 25, 50, 100 and 200 rows, plus 400 rows for classification and the full training set. All models receive the same subsample, and evaluation always uses the complete test split of the seed.

V. RESULTS

A. CLASSIFICATION

Table 4 summarises the breast cancer results. Kumo Tabular obtained the highest mean on all five metrics, followed by logistic regression; the tree ensembles were clearly behind. The largest difference is in recall: pooled over the five test sets, which contain 210 malignant and 360 benign cases in total, Kumo Tabular missed 5 malignant tumours and produced 1 false alarm, compared with 11 and 2 for logistic regression, 14 and 5 for histogram gradient boosting, and 15 and 11 for random forests. Fig. 2 shows the confusion matrices for the first split. ROC-AUC is close to saturation for all models (0.988 to 0.998), so the ROC and precision-recall curves in Fig. 3 separate the models less than recall does.

B. REGRESSION

Table 5 reports the diabetes results. Kumo Tabular again had the best means, but linear regression was close behind on every metric, and both were clearly better than the tree ensembles. Fig. 4 plots predictions against targets for the first split and Fig. 5 the residual distributions. The patterns of Kumo Tabular and linear regression are similar; on that split, their absolute errors had a Pearson correlation of 0.93, compared with 0.76 and 0.72 between Kumo Tabular and the random forest and boosting models. The largest errors are shared across models, which points to noise in the target rather than a weakness of one method.

Predictive intervals. Because Kumo Tabular returns 999 quantiles, it also provides predictive intervals at no extra cost. The nominal 90% interval $[q_{0.05}, q_{0.95}]$ covered $93.3\% \pm 3.3\%$ of test targets on average, with a mean width of 174.2 ± 7.8 (Fig. 6). Coverage ranged from 87.6% to 95.5% across splits, so the intervals are approximately calibrated on these data, slightly conservative on average, and below nominal on one split. None of the baselines provides intervals without additional modelling.

Choice of point estimate. The median minimises expected absolute error, while the mean minimises squared error [40]. Table 6 compares the median with the average over the 999 quantiles, which approximates the predictive mean, using the

TABLE 4. Breast cancer classification results (mean ± standard deviation over five stratified splits, 114 test rows each). Malignant is the positive class. Time is fit plus prediction time per split. Best mean per metric in bold.

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Time (s)
Kumo Tabular	**0.989 ± 0.010**	**0.995 ± 0.010**	**0.976 ± 0.029**	**0.985 ± 0.014**	**0.998 ± 0.003**	1.099 ± 0.012
Logistic Reg.	0.977 ± 0.012	0.990 ± 0.013	0.948 ± 0.031	0.968 ± 0.017	0.995 ± 0.004	0.019 ± 0.001
Random Forest	0.954 ± 0.013	0.947 ± 0.031	0.929 ± 0.024	0.938 ± 0.018	0.988 ± 0.011	1.204 ± 0.063
HistGB	0.967 ± 0.016	0.976 ± 0.024	0.933 ± 0.043	0.953 ± 0.022	0.994 ± 0.007	0.274 ± 0.020

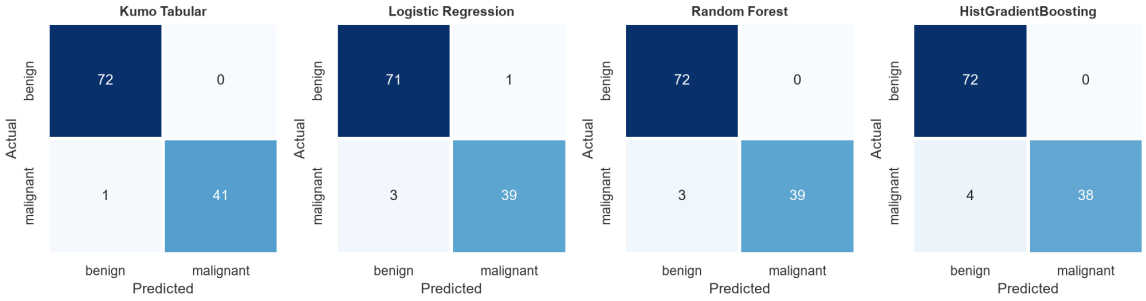


FIGURE 2. Confusion matrices on the first classification split (seed 42, 72 benign and 42 malignant test cases).

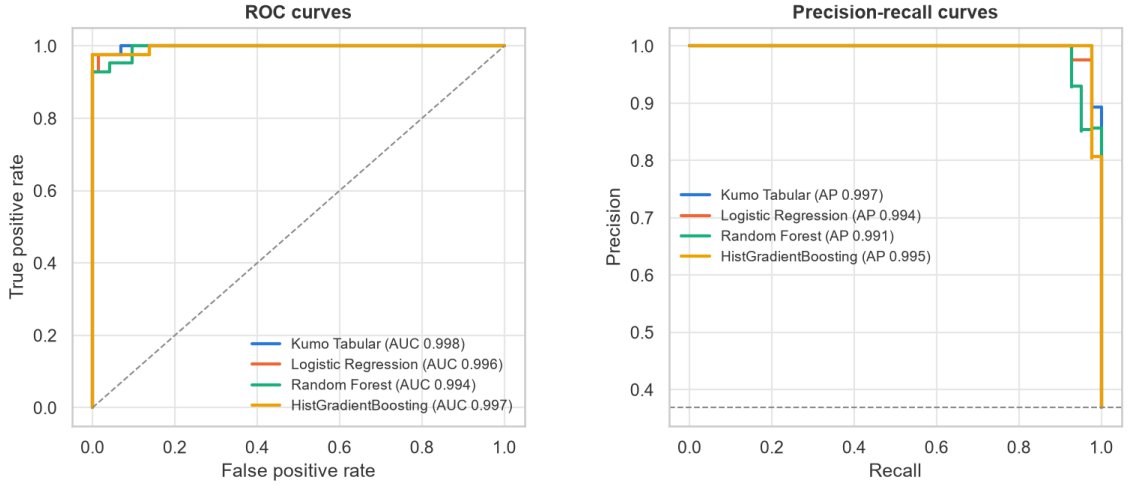


FIGURE 3. ROC curves (left) and precision-recall curves (right) on the first classification split. AP denotes average precision; the dashed lines mark a random classifier.

stored outputs of the same runs. The mean lowered RMSE and raised R^2 slightly, while the median kept the lower MAE, consistent with theory. The median is retained as the reported point prediction.

C. PAIRED STATISTICAL ANALYSIS

Table 7 lists accuracy and R^2 per split, and Table 8 and Fig. 7 summarise the paired comparison with the strongest baseline. In classification, Kumo Tabular was better on all five splits for accuracy, recall and F1, and never worse on any split for any metric. The consistency is clear, but the evidence is weak in formal terms. The naive paired t-test gives $p < 0.05$ for accuracy, recall and F1. After the correction for overlapping training sets, only recall remains below 0.05 ($p_{\text{corr}} = 0.016$, 95% CI $[0.009, 0.048]$), and after Holm adjustment across the five classification metrics no result is significant (smallest adjusted $p = 0.081$). The Wilcoxon test reaches its design minimum of 0.0625 for the three metrics with five wins.

In regression, Kumo Tabular had the lower MAE on four splits but the better RMSE and R^2 on only three, and all corrected intervals include zero by a wide margin. The mean R^2 advantage of 0.007 is small compared with the split-to-split standard deviation of the paired difference, 0.028. On the full diabetes training set, Kumo Tabular and linear regression are therefore statistically indistinguishable in this study.

D. TRAINING-SIZE STUDY

Tables 9 and 10 and Fig. 8 show how performance changes with the number of labelled rows. The two tasks behave differently.

In regression, Kumo Tabular was far more stable than linear regression when data was scarce. With 25 training rows it

TABLE 5. Diabetes regression results (mean ± standard deviation over five random splits, 89 test rows each). Kumo Tabular uses the median quantile $q_{0.5}$ as point prediction. Best mean per metric in bold.

Model	MAE	RMSE	R^2	Time (s)
Kumo Tabular	**42.48 ± 3.10**	**52.88 ± 3.47**	**0.503 ± 0.039**	0.627 ± 0.053
Linear Reg.	43.75 ± 3.59	53.27 ± 4.03	0.496 ± 0.046	0.003 ± 0.000
Random Forest	45.23 ± 2.63	56.16 ± 3.31	0.440 ± 0.022	0.833 ± 0.024
HistGB	45.85 ± 0.72	58.52 ± 1.67	0.390 ± 0.042	0.190 ± 0.031

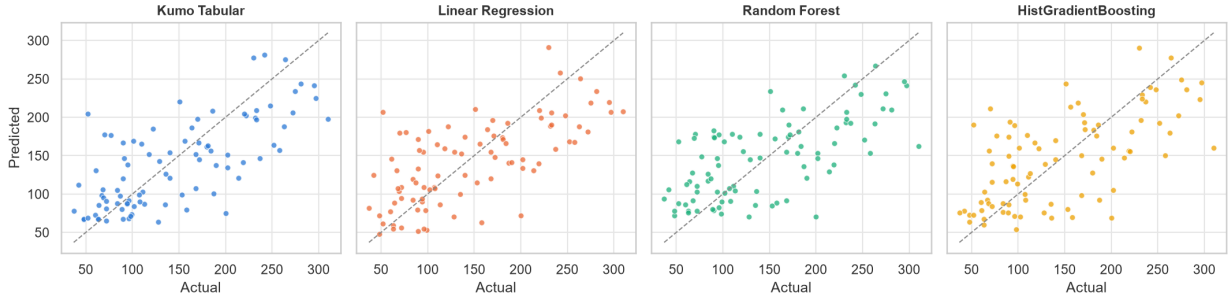


FIGURE 4. Predicted versus actual disease progression on the first regression split (seed 42). The dashed line marks perfect prediction.

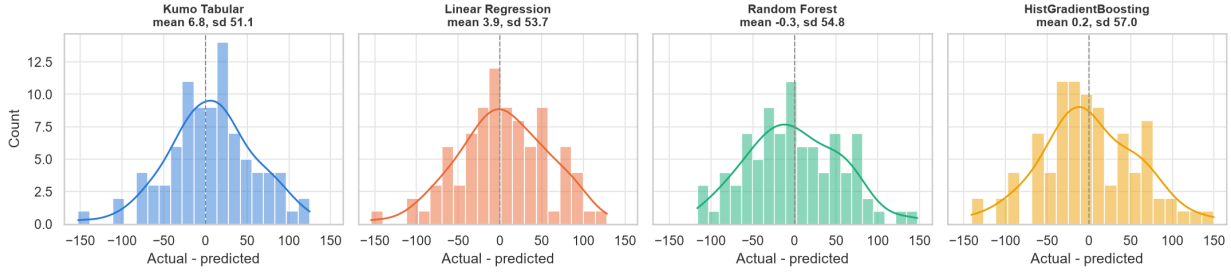


FIGURE 5. Residual distributions (actual minus predicted) on the first regression split, with mean and standard deviation per model.

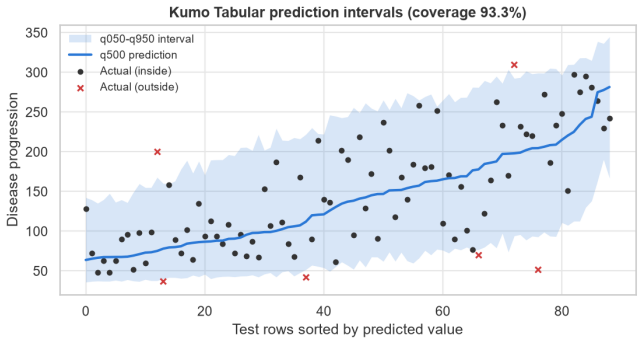


FIGURE 6. Kumo Tabular median prediction and nominal 90% interval for the test rows of the first regression split, sorted by prediction. Crosses mark targets outside the interval.

TABLE 6. Kumo Tabular regression point estimates, averaged over the five splits.

Point estimate	MAE	RMSE	R^2
Median $q_{0.5}$ (reported)	**42.48**	52.88	0.503
Mean of quantiles	42.54	**52.35**	**0.513**

TABLE 7. Per-split results of Kumo Tabular and the strongest baseline. Seeds define the train/test partition shared by all models.

	Accuracy		R^2	
Seed	Kumo	Logistic Reg.	Kumo	Linear Reg.
42	0.991	0.965	0.499	0.453
43	1.000	0.991	0.556	0.542
44	0.974	0.965	0.524	0.525
45	0.991	0.982	0.486	0.519
46	0.991	0.982	0.451	0.441

reached $R^2 = 0.429 \pm 0.058$, compared with 0.075 ± 0.304 for linear regression, which must estimate eleven coefficients from 25 noisy targets, and 0.388 ± 0.127 for random forests. Kumo Tabular remained the best regressor up to 100 rows; at 200 rows it was level with linear regression.

In classification there was no such small-data advantage. Logistic regression slightly exceeded Kumo Tabular at 25 rows (0.956 versus 0.950 accuracy) and the two were level at 50 and 100 rows; Kumo Tabular pulled ahead only from 200 rows onwards. Histogram gradient boosting produced constant predictions at 25 rows (accuracy 0.632, ROC-AUC 0.500), because its default minimum of 20 samples per leaf

TABLE 8. Paired comparison of Kumo Tabular with the strongest baseline over the five splits. $\bar{d}$ is the mean advantage of Kumo (positive favours Kumo for every metric), s_d its standard deviation. The corrected test and interval use the Nadeau–Bengio variance correction for overlapping training sets; Holm adjustment is applied within each task. No result is significant at $\alpha = 0.05$ after adjustment.

Task	Metric	W/T/L	$\bar{d}$	s_d	Corrected 95% CI	p_{naive}	p_{corr}	$p_{\text{corr}}^{\text{Holm}}$	p_{Wilcoxon}
Classification	Accuracy	5/0/0	0.0123	0.0078	[−0.0023, 0.0269]	0.025	0.080	0.313	0.062 (n=5)
	Precision	2/3/0	0.0051	0.0111	[−0.0156, 0.0258]	0.362	0.531	0.535	0.500 (n=2)
	Recall	5/0/0	0.0286	0.0106	[0.0087, 0.0484]	0.004	0.016	0.081	0.062 (n=5)
	F1	5/0/0	0.0172	0.0109	[−0.0031, 0.0376]	0.024	0.078	0.313	0.062 (n=5)
	ROC-AUC	4/1/0	0.0030	0.0034	[−0.0034, 0.0094]	0.126	0.267	0.535	0.125 (n=4)
Regression	MAE	4/0/1	1.27	1.10	[−0.78, 3.32]	0.061	0.161	0.484	0.125 (n=5)
	RMSE	3/0/2	0.39	1.43	[−2.28, 3.07]	0.573	0.704	1.000	0.625 (n=5)
	R^2	3/0/2	0.007	0.028	[−0.046, 0.060]	0.603	0.727	1.000	0.625 (n=5)

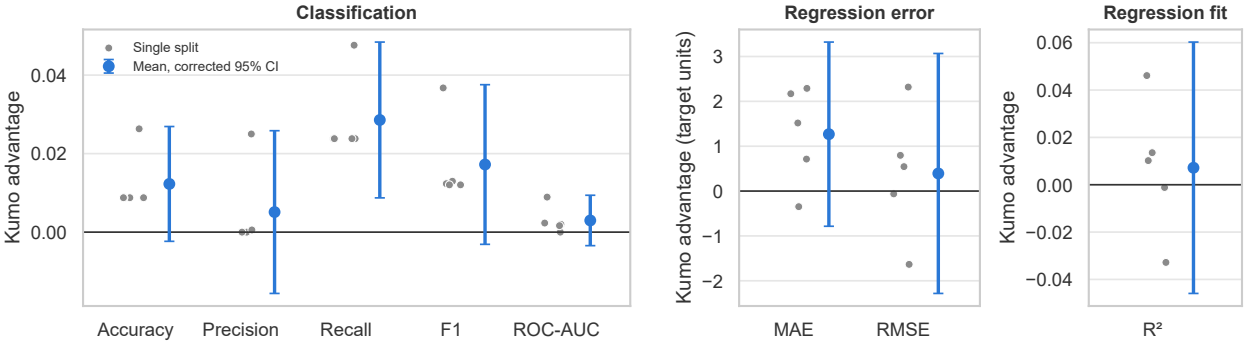


FIGURE 7. Paired advantage of Kumo Tabular over the strongest baseline (logistic regression for classification, linear regression for regression) on each of the five splits (grey) with the mean and the Nadeau–Bengio corrected 95% confidence interval (blue). Positive values favour Kumo Tabular for every metric, including MAE and RMSE.

TABLE 9. Classification accuracy by training-set size (mean ± standard deviation over three seeds; full test split).

n	Kumo Tabular	Logistic Reg.	Random Forest	HistGB
25	0.950 ±0.027	**0.956** ±0.023	0.930 ±0.018	0.632 ±0.000
50	**0.953** ±0.051	**0.953** ±0.022	0.933 ±0.044	0.909 ±0.035
100	**0.956** ±0.040	**0.956** ±0.026	0.939 ±0.015	0.944 ±0.051
200	**0.985** ±0.013	0.971 ±0.013	0.947 ±0.009	0.947 ±0.009
400	**0.982** ±0.015	0.974 ±0.015	0.953 ±0.018	0.956 ±0.009
455	**0.988** ±0.013	0.974 ±0.015	0.959 ±0.013	0.959 ±0.010

TABLE 10. Regression R^2 by training-set size (mean ± standard deviation over three seeds; full test split).

n	Kumo Tabular	Linear Reg.	Random Forest	HistGB
25	**0.429** ±0.058	0.075 ±0.304	0.388 ±0.127	−0.097 ±0.087
50	**0.437** ±0.019	0.331 ±0.137	0.418 ±0.008	0.348 ±0.063
100	**0.486** ±0.045	0.467 ±0.076	0.433 ±0.024	0.410 ±0.043
200	0.505 ±0.085	**0.506** ±0.060	0.438 ±0.057	0.391 ±0.069
353	**0.526** ±0.028	0.507 ±0.048	0.440 ±0.024	0.406 ±0.030

leaves no room for a split. This reflects the default configuration rather than the capability of boosting.

E. RUNTIME

Fig. 9 and the last columns of Tables 4 and 5 compare runtimes. Per split, Kumo Tabular needed 1.10 s for classification and 0.63 s for regression on the GTX 980M, compared with 0.019 s and 0.003 s for logistic and linear regression, roughly 60 and 220 times more. These ratios are sensitive to the very short baseline timings: in a complete re-run of both experiments on the same machine, all metrics were identical, but the per-split times were 1.22 s versus 0.052 s for classification and 0.68 s versus 0.023 s for regression, ratios of about 23 and 30. The published Kaggle notebooks, run on a Tesla T4 GPU with float16 autocast, measured 0.41 to 0.52 s per split for Kumo Tabular and ratios of about 6 to 49 relative to the linear models. The cost ratio therefore depends strongly on hardware and on noise in millisecond-scale timings; across all measured runs it ranged from about 6 to 220. On the GTX 980M, Kumo Tabular was comparable to a 500-tree random forest on the CPU and slower than histogram gradient boosting. In the training-size study, Kumo Tabular prediction time grew from 0.46 s with 25 context rows to 1.11 s with 455 rows for classification, and from 0.44 s to 0.64 s for regression, so a fixed overhead dominates at these sizes. The GPU used here is old and could not use reduced-precision arithmetic; absolute times on current hardware are expected to be lower, but the cost of a forward pass that processes the whole context will remain large relative to applying a fitted linear model.

F. ERROR ANALYSIS

Across the five classification test sets, 38 split-specific test cases were misclassified by at least one model, and 4 of them

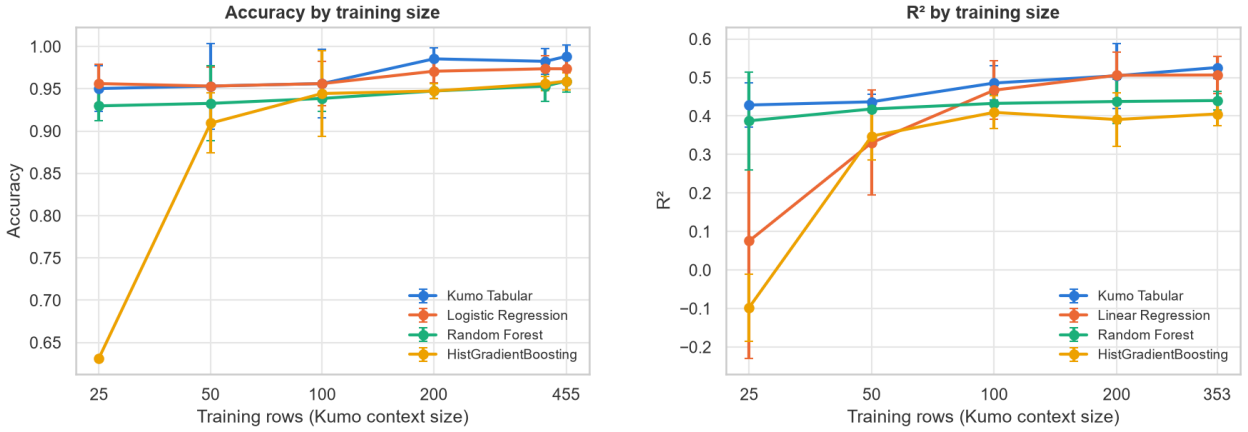


FIGURE 8. Classification accuracy (left) and regression R^2 (right) as a function of the number of training rows, which is also the Kumo Tabular context size. Points show the mean over three seeds, error bars one standard deviation. The horizontal axis is logarithmic; in the left panel the 400-row point is plotted without its own tick label.

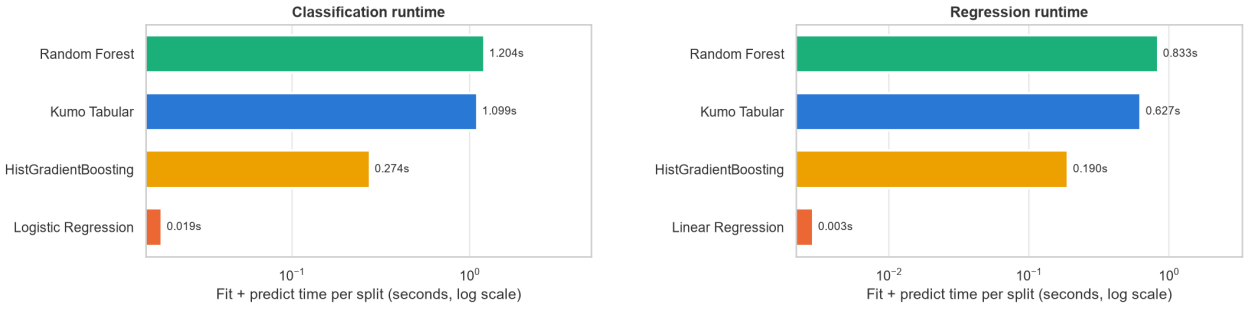


FIGURE 9. Mean fit plus prediction time per split for classification (left) and regression (right), logarithmic scale. Kumo Tabular ran on the GPU, the baselines on the CPU.

by all four models. Kumo Tabular made 6 errors: 4 of these were among the cases that every model misclassified, three of them malignant tumours to which Kumo Tabular assigned a malignancy probability below 0.07; one error was shared only with the random forest; and one was made by Kumo Tabular alone. Most of its remaining errors therefore concern cases that appear genuinely ambiguous in the feature space rather than failures specific to the pretrained model.

VI. DISCUSSION

Where Kumo Tabular helps. Without any tuning or feature engineering, the smallest Kumo Tabular checkpoint matched or exceeded every classical baseline on both tasks. In classification its advantage was consistent across splits and concentrated in recall of the positive class, which is the operationally important error on this dataset. In regression its most useful property was robustness with very few labelled rows, where it avoided the high variance of an unregularised linear model, and its quantile output provided reasonably calibrated intervals for free. These observations agree in direction with reports that pretrained in-context learners are strong on small tables [6], [7].

Where it does not. The size of the advantage was modest. On both full tasks the strongest baseline was a linear model, and the remaining differences are of the order of one test row in classification and within noise in regression. On small classification training sets logistic regression was as accurate as Kumo Tabular. Both datasets are low-dimensional, fully numerical, free of missing values and well suited to linear models; they leave little room for a more expressive model and test none of the capabilities, such as handling categorical features or missing values, that the model was designed for.

Cost of in-context learning. The distinction between (1) and (3) has practical consequences. A classical model concentrates its cost in fitting and then predicts almost instantly; Kumo Tabular has no fitting cost but processes the full context on every prediction call. For a one-off analysis of a small table this trade-off is favourable, since no model selection is needed. For repeated scoring or latency-sensitive use, the per-call cost and the GPU dependency matter, unless context caching, which the architecture supports according to its developers, is used.

Practical implication. For small numerical tables, Kumo Tabular is a reasonable default that is unlikely to perform worse than untuned classical models and can be better when labelled data is scarce. It should not be assumed to outperform a well-chosen linear model by a meaningful margin, and claims of superiority on a specific dataset require more splits

or repetitions than were used here.

VII. THREATS TO VALIDITY

Statistical conclusion validity. Five splits of 114 and 89 test rows provide limited power; one classification error changes accuracy by about 0.009. Resampled splits are dependent, and the corrected test of (10) is itself an approximation [14]. Several metrics were tested per task; the Holm adjustment addresses this within tasks but not across the training-size study, which is reported descriptively only.

Internal validity. All models used identical splits and the implementation guards against target leakage, which is covered by unit tests. Baselines were not tuned, while Kumo Tabular brings tuning-free defaults by design; tuned boosting or regularised linear models could narrow or reverse some differences. Runtimes compare a GPU pipeline with CPU pipelines and are specific to the hardware used.

External validity. Only two small, clean, widely studied datasets were used, with no categorical features and no missing values, and only the smallest of three checkpoints was evaluated. Kumo Tabular was reportedly pretrained on synthetic data only [10], which makes memorisation of these datasets unlikely, but the results cannot be generalised to larger, high-dimensional or heterogeneous tables, to the larger checkpoints, or to other domains.

Construct validity. Threshold-based metrics use a fixed 0.5 threshold; in a diagnostic setting the threshold would be chosen for a target sensitivity. The datasets serve as benchmarks; no conclusion about clinical use is drawn or supported.

VIII. LIMITATIONS AND FUTURE WORK

The main limitations are the small number of datasets and splits, the absence of hyperparameter tuning for the baselines, the evaluation of a single checkpoint size, and runtimes measured on one older laptop GPU. Future work should (i) use repeated cross-validation on a larger and more diverse collection of tables, including categorical features and missing values; (ii) compare against tuned gradient boosting and against other tabular foundation models such as TabPFN [7] and TabICL [8]; (iii) evaluate the medium and large checkpoints and reduced-precision inference on current GPUs; (iv) study probability calibration and threshold selection in classification; and (v) measure the benefit of context caching for repeated predictions.

IX. CONCLUSION

This study compared NVIDIA Kumo Tabular with logistic or linear regression, random forests and histogram gradient boosting on two small public datasets under a controlled and reproducible protocol. Used without tuning, the smallest Kumo Tabular checkpoint achieved the best mean on every metric, never lost to the strongest baseline on any classification split, and was markedly more robust than linear regression with very few training rows. At the same time, its advantage over linear models on the full tasks was small and not statistically significant once the dependence between splits and multiple comparisons were accounted for, it offered no advantage over logistic regression on small classification training sets, and its prediction time per split was about 6 to 220 times that of the linear models, depending on hardware and run. Pretrained in-context learning is therefore a credible and convenient option for small tables, but on these benchmarks it is an incremental rather than a decisive improvement over well-chosen classical models.

DATA AND CODE AVAILABILITY

The implementation, configuration, raw per-split results, figures and the scripts that generate every table and statistic in this paper are available at https://github.com/noyonahmedjoy/Kumo-Tabular [12]. Self-contained notebooks reproducing the experiments are published on Kaggle [13]. Run by the author on Kaggle (Tesla T4 GPU, PyTorch 2.11, float16 autocast), their stored outputs reproduce all reported results for Kumo Tabular, logistic and linear regression and random forests at the precision reported here; results for histogram gradient boosting differ slightly (for example, classification accuracy 0.960 instead of 0.967), presumably because of a different scikit-learn version. Kumo Tabular weights are provided by NVIDIA under the OpenMDW 1.1 licence [9].

ACKNOWLEDGMENT

In accordance with IEEE policy on AI-generated content, the author discloses the use of two AI systems. Claude Opus 5.5 (Anthropic), accessed through the Claude Code desktop application, was used extensively. Working from written specifications provided by the author, it wrote the software in the accompanying repository (data loading, model wrappers, benchmarking, evaluation, plotting, unit tests and notebook generation), executed the local experiments, and produced all figures and tables from the saved experiment outputs, including the diagram in Fig. 1. It performed the statistical analysis described in Section IV-F, searched for and verified the references against bibliographic databases, and drafted the complete text of this article, including the abstract and every section; AI-generated text is therefore not confined to particular sections. ChatGPT (OpenAI; model version not recorded) was used for research planning, for preparing the written specifications given to Claude, for review assistance, and for drafting the author biography. ChatGPT did not generate the experimental results, figures, tables or statistical analyses. The author defined the research topic, objectives, datasets, models and evaluation requirements, ran the published Kaggle notebooks, reviewed the project and the manuscript, and takes full responsibility for the content of this article.

REFERENCES

[1] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Inf. Fusion*, vol. 81, pp. 84–90, 2022.

[2] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2022, arXiv:2207.08815.

[3] V. Borisov, T. Leemann, K. Seßler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 6, pp. 7499–7519, 2024.
[4] D. McElfresh, S. Khandagale, J. Valverde, V. Prasad C, B. Feuer, C. Hegde, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" in *Adv. Neural Inf. Process. Syst. (NeurIPS), Datasets and Benchmarks Track*, 2023, arXiv:2305.02997.
[5] A. Kadra, M. Lindauer, F. Hutter, and J. Grabocka, "Well-tuned simple nets excel on tabular datasets," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021, arXiv:2106.11189.
[6] N. Hollmann, S. Müller, K. Eggensperger, and F. Hutter, "TabPFN: A transformer that solves small tabular classification problems in a second," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2023, arXiv:2207.01848.
[7] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter, "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, no. 8045, pp. 319–326, 2025.
[8] J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan, "TabICL: A tabular foundation model for in-context learning on large data," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, arXiv:2502.05564.
[9] NVIDIA, "Kumo Tabular: Model card," Hugging Face model repository, https://huggingface.co/nvidia/Kumo-Tabular, accessed: Nov. 1, 2026.
[10] J. Qu, V. Hudovernik, M. Jurkovic, C. Lorenz, A. Nitta, D. Gordeev, F. Lopez, R. Bendias, G. Titericz Jr, A. S. Sokolovski, I. Hulseman, J. Leskovec, and M. Fey, "NVIDIA Kumo Tabular sets a new accuracy-efficiency frontier for tabular prediction," Hugging Face community article, https://huggingface.co/blog/nvidia/kumo-tabular, Sep. 2026, accessed: Nov. 1, 2026.
[11] NVIDIA, "structured-data-models, version 0.2.1," Software, https://github.com/NVIDIA/structured-data-models, 2026, accessed: Nov. 1, 2026.
[12] MD. Noyon Ahmed Joy, "Tabular Intelligence: Benchmarking NVIDIA Kumo Tabular against traditional machine learning models," Source code and results, https://github.com/noyonahmedjoy/Kumo-Tabular, 2026, accessed: Nov. 1, 2026.
[13] ——, "NVIDIA Kumo Tabular: Classification benchmark; regression benchmark; Kumo Tabular vs traditional ML: Experiment," Kaggle notebooks, https://www.kaggle.com/code/nayonahmedlol/nvidia-kumo-tabular-classification-benchmark, https://www.kaggle.com/code/nayonahmedlol/nvidia-kumo-tabular-regression-benchmark, https://www.kaggle.com/code/nayonahmedlol/kumo-tabular-vs-traditional-ml-experiment, 2026, accessed: Nov. 1, 2026.
[14] C. Nadeau and Y. Bengio, "Inference for the generalization error," *Mach. Learn.*, vol. 52, no. 3, pp. 239–281, 2003.
[15] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, pp. 65–70, 1979.
[16] D. R. Cox, "The regression analysis of binary sequences," *J. Roy. Statist. Soc. Ser. B*, vol. 20, no. 2, pp. 215–232, 1958.
[17] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
[18] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.
[19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.
[20] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.
[21] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2018, arXiv:1706.09516.
[22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
[23] S. Ö. Arık and T. Pfister, "TabNet: Attentive interpretable tabular learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 8, 2021, pp. 6679–6687.
[24] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2021, arXiv:2106.11959.
[25] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal *et al.*, "Language models are few-shot learners," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, arXiv:2005.14165.
[26] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, arXiv:1706.03762.
[27] S. Garg, D. Tsipras, P. Liang, and G. Valiant, "What can transformers learn in-context? A case study of simple function classes," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2022, arXiv:2208.01066.
[28] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag, "TabLLM: Few-shot classification of tabular data with large language models," in *Proc. Int. Conf. Artif. Intell. Statist. (AISTATS)*, ser. Proc. Mach. Learn. Res., vol. 206, 2023, pp. 5549–5581.
[29] S. Müller, N. Hollmann, S. P. Arango, J. Grabocka, and F. Hutter, "Transformers can do Bayesian inference," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022, arXiv:2112.10510.
[30] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set transformer: A framework for attention-based permutation-invariant neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, arXiv:1810.00825.
[31] T. G. Dietterich and G. Bakiri, "Solving multiclass learning problems via error-correcting output codes," *J. Artif. Intell. Res.*, vol. 2, pp. 263–286, 1995.
[32] R. Koenker and G. Bassett, "Regression quantiles," *Econometrica*, vol. 46, no. 1, pp. 33–50, 1978.
[33] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in *Proc. SPIE 1905, Biomedical Image Processing and Biomedical Visualization*, 1993, pp. 861–870.
[34] B. Efron, T. Hastie, I. Johnstone, and R. Tibshirani, "Least angle regression," *Ann. Statist.*, vol. 32, no. 2, pp. 407–499, 2004.
[35] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.
[36] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019, arXiv:1912.01703.
[37] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Comput.*, vol. 10, no. 7, pp. 1895–1923, 1998.
[38] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bull.*, vol. 1, no. 6, pp. 80–83, 1945.
[39] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, no. 1, pp. 1–30, 2006.
[40] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *J. Amer. Statist. Assoc.*, vol. 102, no. 477, pp. 359–378, 2007.

MD. NOYON AHMED JOY is an independent researcher based in Bangladesh. His research interests include machine learning, tabular data modeling, and empirical evaluation of predictive methods. His work focuses on reproducible experimentation and benchmarking machine learning approaches on structured datasets.